Information Asymmetry as a Structural Condition for Multi-Agent Advantage in Software Engineering Tasks

Kartik Ganapathi*

Abstract

Multi-agent LLM systems can organize work as symmetric peers or hub-and-spoke subagents, but controlled empirical evidence on when each topology produces better outcomes is scarce. We present three experiments comparing peer agents against subagents on software engineering tasks of increasing structural complexity: bug-fixing in a Rust codebase ($n = 24$), feature implementation in a Python library ($n = 24$), and software architecture design with LLM-simulated stakeholders holding asymmetrically distributed requirements ($n = 32$). The first two experiments yield ceiling effects—all treatments achieve perfect quality scores with zero peer-to-peer communication, confirming that code-level tasks without information asymmetry reduce multi-agent coordination to parallelism. The third experiment, which enforces information asymmetry by partitioning stakeholders across agents, produces the first significant result: peers outperform subagents with large effect sizes (composite $d = +0.99$, $p = 0.014$; resolution quality $d = +1.04$, $p = 0.011$). The treatment advantage for conflict resolution is approximately 3.5 times larger when 75% of conflicts span the partition boundary compared to 25%, supporting a directional dose-response pattern between cross-agent information dependency and output quality. Despite the peer messaging channel being available, agents never use it directly across any experiment; all inter-agent information flow is coordinator-mediated, suggesting the advantage stems from the peer topology—long-lived agents with shared file context and iterative relay—rather than explicit collaboration. Two independent evaluation methods (a rubric-based assessment and a blind architectural review) converge on the treatment advantage while measuring complementary qualities. Information asymmetry, not task difficulty, is the structural condition that determines whether multi-agent coordination improves output quality beyond parallelism.[1]

1 Introduction

Multi-agent LLM systems have proliferated rapidly. Frameworks such as AutoGen [10], MetaGPT [5], and CrewAI [8] enable developers to decompose complex tasks across multiple LLM instances, and product features such as Claude Code's Agent Teams [2] bring multi-agent coordination to interactive coding assistants. The premise is intuitive: multiple specialized agents should outperform a single generalist, just as cross-functional teams outperform individual engineers on complex projects.

Yet rigorous empirical evidence for this premise is thin. Most evaluations compare multi-agent systems against single-agent baselines on benchmark suites [6], not against alternative multi-agent topologies on the same task. The question is not whether multiple agents can solve problems—they clearly can—but *when a*

*Independent Researcher. Contact: gkartik@gmail.com

[1]Code, data, and experiment protocols: https://github.com/kar-ganap/ate-series

particular multi-agent topology produces measurably better outcomes than an alternative topology with the same number of agents and the same information access.

We focus on two topologies that represent the practical design space:

- **Hub-and-spoke (subagents):** A coordinator dispatches subtasks to worker agents via a task tool. Workers execute, return reports, and terminate. Cross-agent information flows only through the coordinator.
- **Symmetric peers (agent teams):** Multiple agents operate concurrently with direct communication channels, shared filesystem access, and persistent context. No strict hierarchy.

This paper presents three experiments that progressively identify the structural condition under which peer agents outperform subagents. The first two experiments—bug-fixing in a 500K+ LOC Rust codebase and feature implementation in a Python ML framework—produce ceiling effects: both topologies achieve identical quality with zero inter-agent communication. These informative negatives reveal that when every agent can independently discover all necessary information, coordination adds no quality benefit beyond parallelism (speedups of 3.5–5.2×).

The third experiment changes the structure. We design an architecture task where six LLM-simulated stakeholders hold private, non-overlapping requirements. Each agent interviews only its assigned stakeholders, creating *information asymmetry by construction*. Conflicts that span the agent boundary require cross-agent information transfer to resolve well. Under this condition, peer agents significantly outperform subagents ($d = +0.99$, $p = 0.014$), and the advantage for conflict resolution quality is 3.5× larger when 75% of conflicts cross the partition boundary compared to 25%.

A surprising finding threads through all three experiments: agents never use direct peer messaging, despite the channel being available. All inter-agent information flow is coordinator-mediated. The treatment advantage arises from the *topology*—long-lived agents, shared file context, and iterative relay—not from explicit collaboration. This reframes the practical recommendation: design for information distribution, not communication protocols.

2 Background and Related Work

2.1 System Under Study

All experiments use Claude Code, Anthropic's interactive coding assistant [2]. In its default configuration, Claude Code provides a *Task tool* that spawns subagent processes: the lead agent dispatches a task description, the subagent executes autonomously, and returns a result. Subagents do not persist after task completion and cannot communicate with each other.

With the Agent Teams feature enabled (`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`), Claude Code spawns *symmetric peer agents* that share a filesystem, maintain persistent context, and can communicate via a `SendMessage` tool. A coordinator process manages agent lifecycle, but peers operate without strict hierarchy. Anthropic's case study on building a C compiler with parallel agents [1] demonstrated that peer agents can coordinate on complex tasks, but did not compare against the hub-and-spoke alternative.

The agent model for all experiments is Claude Opus 4.6. Stakeholder simulation (Experiment 3 only) uses Claude Haiku 4.5 at temperature 0.2. Rubric scoring uses Claude Sonnet 4.6; blind architectural review uses Claude Opus 4.6.

Table 1. Preliminary experiment summary.

	Experiment 1	Experiment 2
Domain	Bug-fixing (Ruff, Rust)	Feature impl. (LangGraph, Python)
Tasks	8 bugs	8 features (104 tests)
Treatments	3	3
Quality result	8/8 all treatments	104/104 all treatments
Max speedup	5.2×	3.5×
Peer messages	0	0

2.2 Related Work

Multi-agent frameworks. AutoGen [10] provides conversable agents with flexible topologies. MetaGPT [5] assigns software engineering roles (PM, architect, engineer) to distinct agents. CrewAI [8] structures agents into "crews" with defined roles and tools. These frameworks primarily evaluate end-to-end task success rather than comparing topologies on the same task.

LLM-as-judge. Zheng et al. [11] establish that strong LLMs can serve as reliable evaluators for open-ended generation tasks, though concerns about self-evaluation bias persist. Our dual-evaluation design (Section 5.3) is motivated by the need to triangulate automated assessments.

Information processing in organizations. Galbraith's [4] information processing view of organizations posits that coordination costs rise with task uncertainty and information distribution. Thompson's [9] typology of interdependence (pooled, sequential, reciprocal) predicts that reciprocal interdependence—where agents must iteratively exchange information—demands the most coordination. Our experimental design operationalizes this by varying the proportion of conflicts that require cross-agent information.

3 Preliminary Experiments

Before designing the primary experiment, we conducted two preliminary studies on real-world software engineering tasks to test whether peer agents outperform subagents on code-level work. Both tasks were drawn from active open-source projects, and all patches were verified against existing test suites (Table 1).

Experiment 1: Bug-fixing. Eight bugs from the Ruff Python linter (Rust, 500K+ LOC) were selected from GitHub issues confirmed unfixed at a pinned commit. Three treatment conditions were executed across all eight bugs ($n = 24$ runs total): single-agent control, hub-and-spoke subagents, and symmetric peer agents. Every treatment achieved a perfect 8/8 solve rate. Zero peer-to-peer messages were observed. The peer topology provided a 5.2× wall-clock speedup through parallelism, but no quality improvement.

Experiment 2: Feature implementation. Eight features for the LangGraph Python library were specified via detailed feature documents. A tiered acceptance test suite (104 tests across four tiers) provided automated quality measurement. Three treatments were executed ($n = 24$ runs total). All treatments passed 104/104 tests—a perfect ceiling. Again, zero peer-to-peer messages. The peer topology achieved a 3.5× speedup.

The pattern across both experiments is consistent: when every agent has full access to the codebase and each task is independently solvable, the peer topology provides no quality advantage. The tasks lack *information asymmetry*—there is nothing one agent knows that another cannot independently discover. This motivated the design of Experiment 3, which enforces information asymmetry by construction.

Table 2. Stakeholder summary for the DataFlow Corp scenario.

Stakeholder	Role	Key Concerns
Elena Vasquez	Chief Security Officer	End-to-end encryption, zero-trust IAM, key rotation
Marcus Chen	Compliance Lead	GDPR data residency, pseudonymization, audit trails
Sophie Muller	EU Regional Ops	Query latency SLA (<200ms p99), EU-only failover
Kenji Tanaka	APAC Regional Ops	Batch processing windows, burst traffic, regional autonomy
David Okonkwo	Platform Architect	Unified data model, schema evolution, metadata service
Aisha Patel	Product Manager	Dashboard latency (<500ms), feature velocity, analytics

4 Experiment Design

4.1 Research Question and Hypotheses

The preliminary experiments suggest that multi-agent quality advantage requires a structural condition absent in code-level tasks. Experiment 3 tests this hypothesis directly. We hypothesize that this condition is *information asymmetry*: when agents hold non-overlapping information, cross-agent communication becomes necessary (not merely available) for high-quality synthesis.

H1 (Architecture effect): When agents can only access disjoint subsets of stakeholders, peer agents produce higher-quality architecture documents than hub-and-spoke subagents.

H2 (Dose-response): The peer advantage grows as more conflicts span the agent partition boundary (from Partition A to Partition C).

H3 (Communication utility): In treatment runs, the quality of inter-agent information transfer correlates with output quality.

4.2 Domain: Architecture Design with Simulated Stakeholders

We designed a scenario ("DataFlow Corp") in which a company must build a multi-region data platform spanning the US, EU, and APAC. Six stakeholders (Table 2) hold private constraint sheets containing hard constraints, preferences, and hidden dependencies. Stakeholders are simulated by Claude Haiku 4.5 (temperature 0.2) and do not volunteer information unprompted—agents must ask specific questions to elicit requirements.

The scenario contains 8 conflicts arising from genuine tensions between stakeholders (e.g., encryption requirements vs. latency SLAs, compliance timelines vs. batch processing windows) and 4 hidden dependencies that are only revealed when agents ask sufficiently targeted questions.

This domain breaks the ceiling observed in Experiments 1–2 because: (a) solution spaces are broad (no single correct architecture), (b) information is distributed by construction (each agent interviews only its assigned stakeholders), and (c) resolution quality depends on synthesizing requirements from both sides of the partition.

4.3 Architecture Conditions

Both conditions use two agents plus a coordinator/lead (Figure 1). The structural difference is how cross-agent information flows.

Control (hub-and-spoke): The lead agent dispatches subagents via the Task tool. Each subagent interviews its assigned stakeholders, returns a report, and terminates. The lead cannot interview stakeholders directly—it

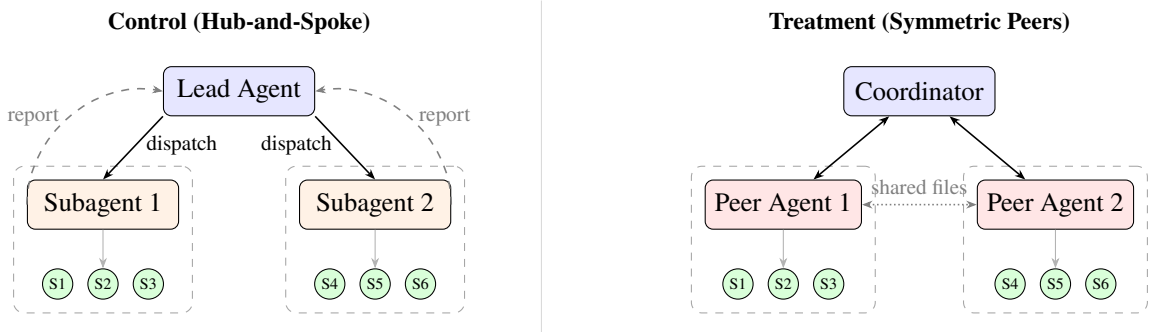

Figure 1. Architecture conditions. **Left:** Hub-and-spoke—lead dispatches subagents that interview stakeholders, return reports, and terminate. **Right:** Symmetric peers are long-lived, share a filesystem, and receive coordinator-mediated relay. Dashed boxes group each agent with its assigned stakeholders. Both conditions use two agents plus a coordinator/lead.

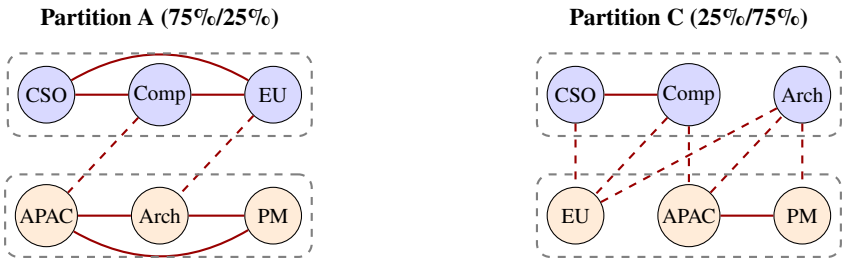

Figure 2. Partition conditions showing conflict graph. Solid lines: within-partition conflicts (resolvable by one agent). Dashed lines: cross-partition conflicts (require both agents' information). **Left:** Partition A has 6 within / 2 cross. **Right:** Partition C has 2 within / 6 cross.

must synthesize from subagent reports alone. Cross-partition information flows serially: Agent 1 → Lead → Agent 2 → Lead → synthesis.

Treatment (symmetric peers): Agent Teams enabled. Two peer agents each interview their assigned stakeholders. Peers can communicate via `SendMessage` and co-edit shared files. Both agents are long-lived and can conduct follow-up interviews informed by the other agent's discoveries. The coordinator manages agent lifecycle but does not synthesize independently.

4.4 Partition Conditions

The six stakeholders form a conflict graph with natural clusters (Figure 2). By reassigning stakeholders across the agent boundary, we vary how many of the 8 conflicts require cross-agent information.

Partition A (low cross-dependency): Agent 1 receives {Security Officer, Compliance Lead, EU Ops}; Agent 2 receives {APAC Ops, Platform Architect, Product Manager}. This yields 6 within-partition and 2 cross-partition conflicts (75/25 split). Most conflicts are resolvable by one agent alone.

Partition C (high cross-dependency): Agent 1 receives {Security Officer, Compliance Lead, Platform Architect}; Agent 2 receives {EU Ops, APAC Ops, Product Manager}. This yields 2 within-partition and 6 cross-partition conflicts (25/75 split). Most conflicts require both agents' information.

Partition B (50/50) was designed but omitted after piloting: with $n = 8$ per cell, a predictable midpoint adds less statistical value than deepening each extreme.

Table 3. Experimental matrix.

Cell	Architecture	Partition	Within/Cross	n
Control-A	Hub-and-spoke	A	6/2 (75%/25%)	8
Control-C	Hub-and-spoke	C	2/6 (25%/75%)	8
Treatment-A	Symmetric peers	A	6/2 (75%/25%)	8
Treatment-C	Symmetric peers	C	2/6 (25%/75%)	8
Total				**32**

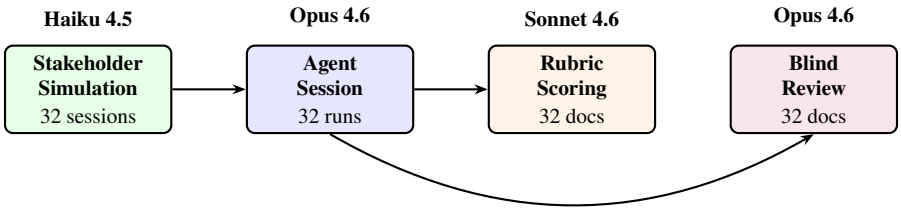

Figure 3. Evaluation pipeline. Each stage uses a different Claude model. Stakeholders are simulated by Haiku 4.5 (temperature 0.2), agents are Opus 4.6, rubric scoring uses Sonnet 4.6, and the blind review uses a separate Opus 4.6 instance with no rubric knowledge.

4.5 Execution

Table 3 summarizes the experimental matrix. All 32 runs were conducted as interactive Claude Code sessions using Claude Opus 4.6 as the agent model. A 30-minute soft time cap was applied. Each run was self-contained with no cumulative state. Runs were executed under a Claude Max subscription ($0 marginal API cost for agent sessions); stakeholder simulation costs totaled approximately $1–2.

5 Evaluation Methodology

We employ two independent evaluation methods to mitigate single-method bias: a rubric-based assessment with ground-truth items and a blind architectural review with no rubric knowledge. Figure 3 shows the model pipeline: each stage uses a different Claude model to reduce self-evaluation bias.

5.1 Rubric-Based Evaluation

A four-layer rubric scores each architecture document against a ground truth that defines the optimal resolution for each conflict and dependency:

- **L1 – Constraint Discovery** (weight 0.25): Did the document mention each stakeholder's hard constraints? Scored as a checklist (found/not found) via semantic matching.
- **L2 – Conflict Identification** (weight 0.25): Did the document identify the known conflicts between stakeholders? Checklist scoring via semantic matching.
- **L3 – Resolution Quality** (weight 0.30): For each identified conflict, how well was it resolved? Scored categorically: *Optimal* (1.0)—satisfies both stakeholders' hard constraints with minimal preference compromise; *Acceptable* (0.67)—satisfies hard constraints but with significant preference trade-offs; *Poor* (0.33)—violates one or more hard constraints; *Missing* (0.0)—conflict not addressed. Scored by Claude Sonnet 4.6 as LLM-as-judge.
- **L4 – Hidden Dependencies** (weight 0.20): Did the document discover non-obvious cross-stakeholder dependencies? Checklist scoring via semantic matching.

The composite score is the weighted sum: $\text{Composite} = 0.25 \cdot L1 + 0.25 \cdot L2 + 0.30 \cdot L3 + 0.20 \cdot L4$.

5.2 Blind Architectural Review

All 32 architecture documents were independently reviewed by Claude Opus 4.6 with zero knowledge of the rubric, ground truth, or experimental conditions. The reviewer scored each document on six dimensions using a 1–5 scale (total /30):

1. **Internal Consistency (IC):** Do component decisions compose without contradictions?
2. **Specificity (SP):** Concrete technology choices vs. hand-wavy prose?
3. **Buildability (BU):** Could an engineering team implement this as written?
4. **Trade-off Depth (TD):** Quality of conflict reasoning?
5. **Completeness (CO):** Operational concerns, edge cases, fallback strategies?
6. **Architectural Soundness (AS):** Do patterns fit the problem domain?

Documents were reviewed in four batches of eight, with one document per cell per batch for calibration consistency.

5.3 LLM-as-Judge: Validity Considerations

A core concern is that Claude models evaluate artifacts produced by Claude agents. The scoring pipeline involves Claude Opus 4.6 agents producing documents, Claude Sonnet 4.6 scoring the rubric, and Claude Opus 4.6 conducting the blind review. Three design features mitigate this concern.

First, *dual-evaluation divergence*: if a systematic bias favoring treatment outputs existed (e.g., stylistic familiarity), both evaluation methods would converge on the same individual champions. They do not. In three of four cells, the rubric champion and the blind review champion are different runs (Section 6.5). The methods agree on the *direction* of the treatment effect but disagree on which individual documents are best, suggesting they measure genuinely different qualities.

Second, Layers L1, L2, and L4 of the rubric are essentially *checklist-based*: does the document mention item X? This reduces the subjective latitude available to the scorer. Only L3 (resolution quality) requires open-ended judgment.

Third, the blind review was *uninstructed*: the reviewer received no rubric, no ground truth, and no scoring criteria. The six dimensions emerged from the reviewer's own assessment framework, providing an independent signal.

We acknowledge that these mitigations do not eliminate the possibility of systematic bias. An independent human evaluation would further strengthen the findings.

6 Results

6.1 Architecture Main Effect

Table 4 reports the architecture main effect (Treatment vs. Control, pooled across partitions). Mann-Whitney U tests are used throughout due to the ordinal nature of rubric scores and small sample sizes [7]. Effect sizes are Cohen's d [3].

L1 (Constraint Discovery) shows a ceiling in all four cells (≥ 0.98), confirming that both topologies effectively elicit basic stakeholder requirements. This L1 ceiling serves as a manipulation check: both topologies successfully extract stakeholder information, confirming that differences in L2–L4 arise from how agents *synthesize* across stakeholders, not from differential information access. The treatment advantage concentrates in L2–L4 (Figure 4): the layers that require cross-stakeholder information synthesis. The largest effect is in

Table 4. Architecture main effect (Treatment vs. Control, $n = 16$ each).

Metric	Control	Treatment	U	p	d
Composite	0.63 ± 0.17	0.80 ± 0.18	62.0	0.014*	+0.99
L2 Conflict ID	0.53 ± 0.30	0.79 ± 0.28	63.0	0.013*	+0.88
L3 Resolution	0.41 ± 0.18	0.64 ± 0.26	60.0	0.011*	+1.04
L4 Hidden Deps	0.64 ± 0.33	0.83 ± 0.27	83.5	0.077†	+0.62

$^{*}p < 0.05$ $^{\dagger}p < 0.10$

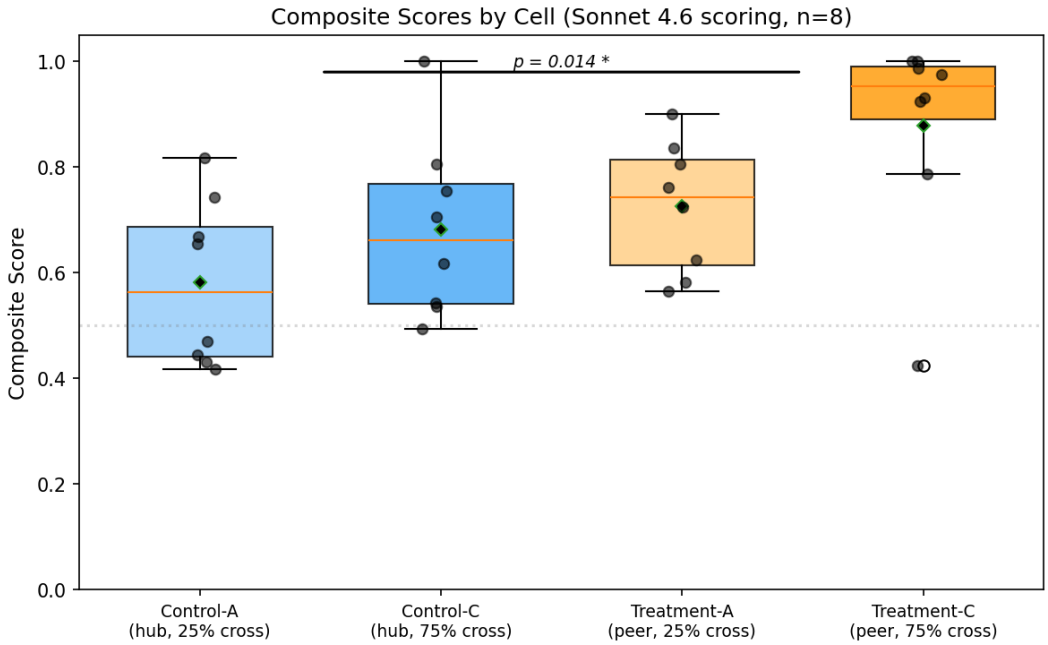

Figure 4. Composite score distributions by experimental cell. Treatment conditions (right) consistently outperform their control counterparts, with the largest separation in Partition C.

L3 (Resolution Quality, $d = +1.04$, $p = 0.011$)—the most demanding layer, requiring agents to synthesize conflicting requirements into a coherent architectural resolution. The widest pairwise contrast, Control-A vs. Treatment-C, yields $d = +1.67$ (composite) and $d = +2.13$ (L3)—the full range of the experimental design from lowest information asymmetry with hub-and-spoke to highest information asymmetry with peer agents.

6.2 Dose-Response

Table 5 decomposes the treatment effect by partition condition. If information asymmetry is the mechanism, the treatment advantage should be larger in Partition C (75% cross-partition conflicts) than in Partition A (25%).

The L3 dose-response pattern is striking (Figure 5): the treatment advantage for resolution quality is 3.5× larger in Partition C (+0.36) than in Partition A (+0.10). The interaction term ($p = 0.129$) does not reach conventional significance at $n = 8$ per cell, but the gradient is monotonic and directionally consistent with H2. Power analysis suggests $n \approx 12$–16 per cell would be needed to detect this interaction at $\alpha = 0.05$.

L2 (Conflict Identification) shows a *flat* gradient: the treatment advantage for identifying conflicts is similar regardless of partition. This is consistent with the mechanism—both topologies can identify conflicts, but *resolving* cross-partition conflicts specifically benefits from the peer topology's information flow.

Table 5. Dose-response: treatment effect by partition condition.

Metric	Effect in A	Effect in C	Gradient	Interaction p
Composite	+0.14	+0.20	+0.05	0.690
L2	+0.28	+0.23	−0.05	0.887
L3	+0.10	+0.36	+0.25	$0.129^{\dagger}$

Interaction p from 100,000-permutation test. $^{\dagger}p < 0.15$, directionally consistent with H2.

Table 6. Blind review per-dimension means (1–5 scale).

Dimension	Control	Treatment	Δ
Internal Consistency	4.00	3.94	−0.06
Specificity	3.31	4.06	+0.75
Buildability	3.13	3.56	+0.44
Trade-off Depth	4.00	4.31	+0.31
Completeness	3.88	4.19	+0.31
Architectural Soundness	3.94	3.88	−0.06

6.3 Resolution Quality Breakdown

Figure 6 shows the per-run L3 scores. Treatment-C achieves L3 ≥ 0.88 in 5 of 8 runs, a level of resolution quality never observed in any other cell (except a single outlier, Control-C-7, at 1.00). Treatment-C runs 1–3 average 7.0 *Optimal* resolutions out of 8 conflicts, compared to 1–2 *Optimal* in Control cells.

Treatment-C also exhibits the highest variance (SD 0.27 on L3). When peer coordination works well, it works exceptionally (L3 ≥ 0.88 in 5/8 runs). When it fails, output degrades below the control floor (runs 4–5: L3 = 0.67 and 0.21). The hub-and-spoke topology trades ceiling for consistency.

6.4 Blind Architectural Review

The blind review confirms the rubric's direction (Table 6). Treatment documents score 23.9 ± 2.2 out of 30, compared to 22.3 ± 1.2 for control (Δ = +1.7).

The treatment advantage concentrates in three dimensions: *Specificity* (+0.75)—treatment documents name concrete technologies, provide latency budgets, and specify deployment topologies where control documents hedge with "or equivalent"; *Buildability* (+0.44)—treatment documents are sprint-ready while control documents require another design round; and *Completeness* (+0.31)—treatment documents surface more hidden dependencies and edge cases. Internal Consistency and Architectural Soundness show ceiling effects at ∼4/5, providing no discrimination.

6.5 Cross-Validation of Evaluation Methods

The rubric and blind review measure different qualities (Figure 7) and pick different individual champions in three of four cells. For example, Control-C-7 achieves a perfect rubric composite of 1.00 (every conflict identified and optimally resolved) but scores only 22/30 on the blind review—a checklist-compliant document that lacks engineering specificity. Conversely, Treatment-C-4 scores 0.79 on the rubric but achieves 27/30 on the blind review (tied highest in the experiment)—a detailed, buildable document that the rubric undervalues because its strength lies in dimensions the rubric does not capture.

Both methods agree that Treatment-C is the best cell and Control-A is the worst. This convergence on direction, combined with divergence on individual rankings, strengthens confidence that the treatment effect

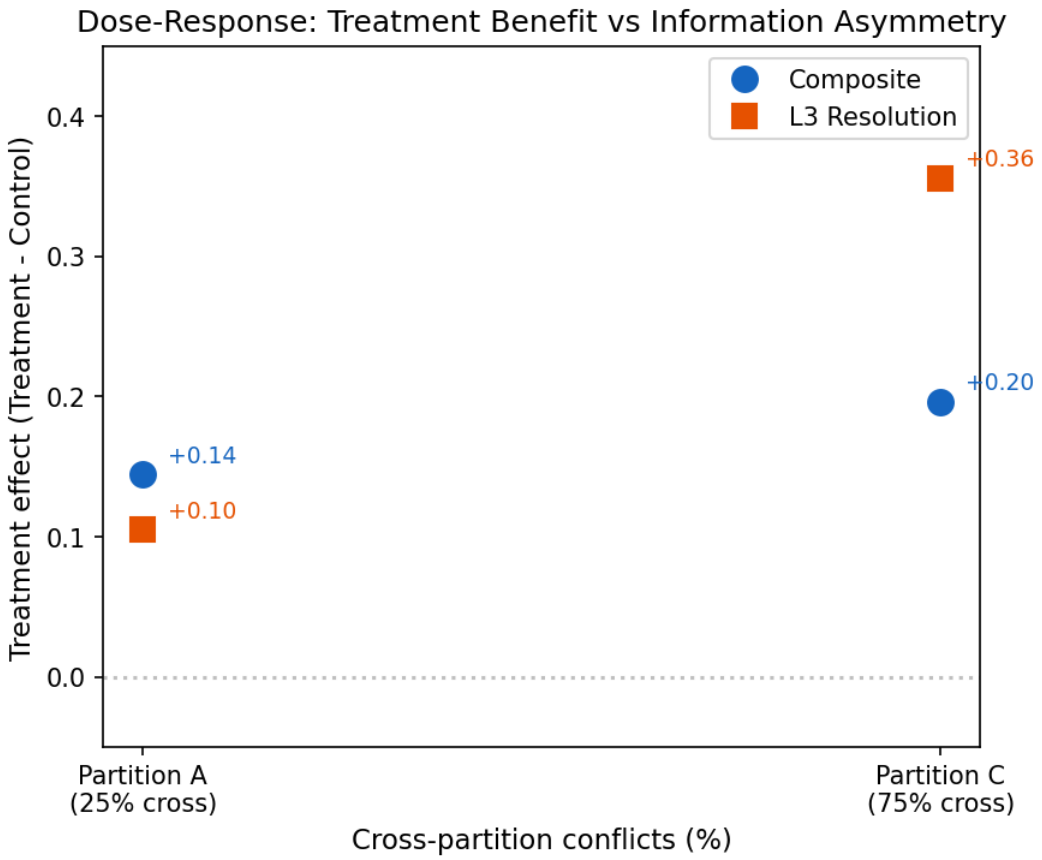

Figure 5. Dose-response gradient. The treatment advantage in L3 (Resolution Quality) is 3.5× larger in Partition C (high cross-partition conflict density) than in Partition A, consistent with an information-asymmetry mechanism.

is not an artifact of either evaluation method.

7 Discussion

7.1 The Zero-Direct-Messages Finding

Across all three experiments—24 bug-fixing runs, 24 feature implementation runs, and 32 architecture design runs—agents *never* used the `SendMessage` tool to communicate directly with a peer agent. All inter-agent information flow in treatment runs was coordinator-mediated: an agent reports findings to the coordinator, the coordinator relays relevant information to the other agent. We reconstructed these mediated exchanges by temporal adjacency and directionality in the transcript (Agent 1 → Coordinator immediately followed by Coordinator → Agent 2).

Yet the treatment significantly outperforms the control. This implies the advantage is *topological*, not *behavioral*:

1. **Long-lived agents:** Peer agents persist throughout the session and can be re-queried. Subagents terminate after returning a report, preventing iterative refinement.
2. **Shared file context:** Both peer agents read and edit the same architecture document. Agent 2 can respond to Agent 1's contributions in real time, without the coordinator explicitly relaying them.
3. **Iterative cross-pollination:** The coordinator relays findings between peers, and peers conduct follow-up stakeholder interviews informed by the other agent's discoveries. This creates a feedback loop absent in

Figure 6. L3 (Resolution Quality) per-run breakdown. Treatment-C achieves ≥ 0.88 in 5/8 runs—a level never reached by other cells except a single outlier (Control-C-7). Treatment-C also shows the highest variance, reflecting the bimodal outcome distribution of peer coordination.

Table 7. Communication metrics for treatment cells.

Cell	Relay events	Mean similarity	Indirect collab.	Msg range
Treatment-A	3/8 runs	0.092	3/8 runs	2–13
Treatment-C	4/8 runs	0.276	3/8 runs	3–12

the control, where subagents' follow-up questions are not informed by the other subagent's findings.

Three communication metrics (Table 7) provide mechanistic evidence for why the peer topology outperforms. *Relay transparency* measures whether information from one agent's stakeholder interviews appears in the other agent's outputs, quantified as the cosine similarity between interview content embeddings and cross-agent statements. *Indirect collaboration* detects whether both agents read or modified the same file (from transcript file operations). *Message volume* counts coordinator-mediated messages per run.

Treatment-C shows 3× higher relay similarity than Treatment-A (0.276 vs. 0.092), indicating that when cross-partition information is relayed in the high-cross condition, it is relayed with greater fidelity. The four Treatment-C runs with detected relay events are also the four highest-scoring runs in that cell (composite 0.93–1.00, relay similarity 0.10–0.42), suggestive of a quality–fidelity relationship though the sample ($n = 4$) precludes formal testing.

Message volume does *not* predict quality: treatment-C-3 (5 messages, composite 1.00) outperforms treatment-A-7 (13 messages, composite 0.73). The advantage comes from what is relayed, not how much.

Indirect collaboration detection rates are low (3/8 in both cells), but this likely underestimates the true rate: Agent Teams transcripts do not expose agent-level attribution for file operations, so all edits appear coordinator-level. The shared file context mechanism described above likely operates in most or all treatment runs.

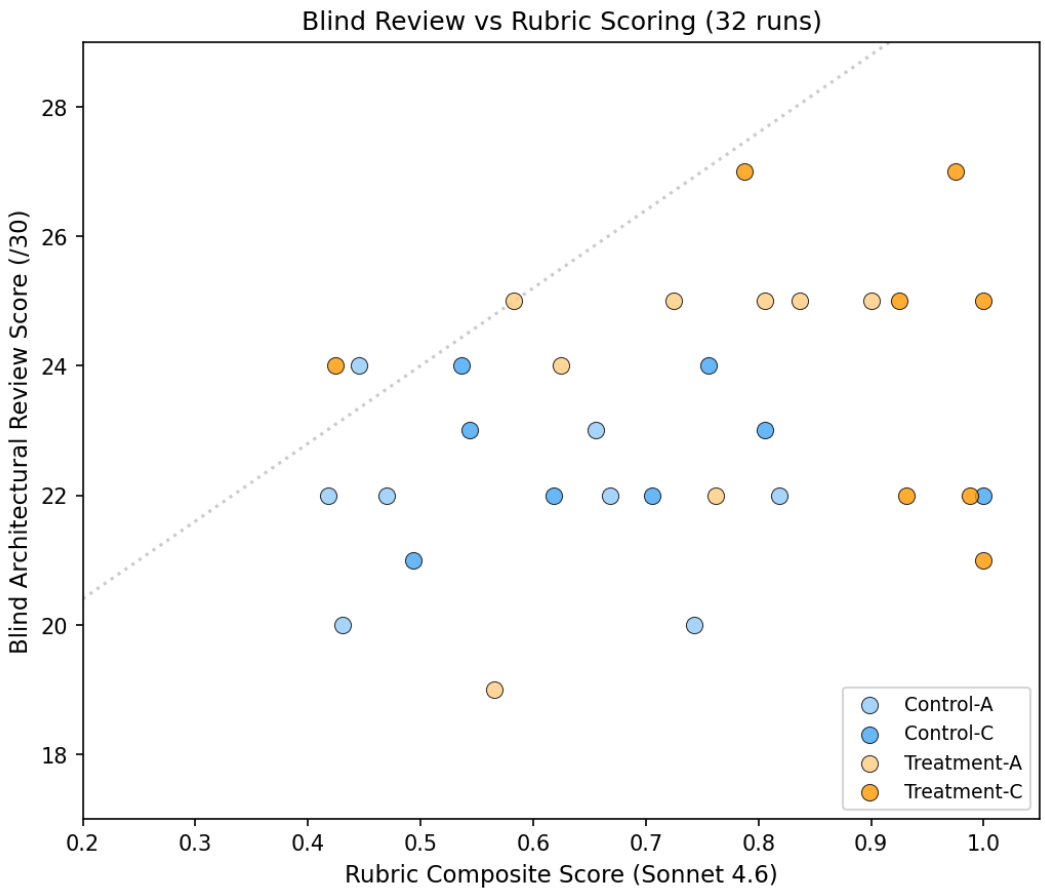

Figure 7. Cross-validation between rubric-based and blind review evaluations. Both methods agree on cell-level direction (Treatment-C best, Control-A worst) but select different individual champions, indicating complementary rather than redundant evaluation.

Treatment runs take 60–75% longer (Table 8) and involve roughly twice the stakeholder interviews. This additional process is the mechanism, not a confound: the peer topology enables a deeper exploration process (discovery → relay → informed follow-up → synthesis) that the hub-and-spoke architecture structurally cannot support. Crucially, additional time alone cannot replicate this process under hub-and-spoke: subagents terminate after returning reports, so the lead cannot dispatch informed follow-up interviews—it can only re-dispatch fresh subagents that lack the other agent's context.

The implication for multi-agent system design is that *channel availability does not guarantee channel use*. Agents never spontaneously adopt lateral communication. The advantage comes from the topology's architectural properties, not from emergent collaborative behavior.

7.2 What Information Asymmetry Buys

The three experiments form a natural ablation study:

- Experiments 1–2: no information asymmetry → no quality advantage (ceiling effect, zero communication).
- Experiment 3: information asymmetry by construction → significant quality advantage ($d = +0.99$).

The distinction is not task difficulty. Architecture design is not inherently "harder" than feature implementation—it is *structurally different* because no single agent holds the full picture. This is the default condition in real organizations: different teams hold different knowledge, and high-quality synthesis requires cross-boundary

Table 8. Process metrics by cell (mean ± SD).

Cell	Wall clock (min)	Interviews	Peer msgs	Unique pairs
Control-A	8.8 ± 1.2	8.4 ± 1.5	—	—
Control-C	10.0 ± 2.0	13.6 ± 3.8	—	—
Treatment-A	14.6 ± 3.3	16.4 ± 5.8	6.8 ± 3.8	2.1 ± 0.4
Treatment-C	15.6 ± 1.8	16.6 ± 4.7	5.8 ± 2.7	2.3 ± 0.5

information transfer. Multi-agent systems should be evaluated on tasks that reflect this structure.

7.3 Implications for Practitioners

1. **Use peer topologies when agents hold asymmetric information.** If all agents have full access to the same data, subagents are sufficient for parallelism.
2. **Do not expect agents to use messaging channels.** In 80 runs across three experiments, agents never spontaneously sent peer messages. If structured communication is needed, build it into the protocol, not the channel.
3. **The time cost is real.** Treatment runs take 60–75% longer. The quality gain must justify the time cost for a given use case.
4. **Invest in information design.** The biggest lever for multi-agent quality is how information is distributed across agents, not how communication channels are configured.

7.4 Limitations

Single scenario. All 32 runs use the DataFlow Corp scenario. Results may not generalize to other domains or conflict structures.

Single model. All agents are Claude Opus 4.6. The treatment effect may differ for other model families or capability levels.

LLM-as-judge. Despite dual-evaluation divergence (Section 5.3), we cannot fully rule out systematic bias from Claude evaluating Claude-generated artifacts. An independent human evaluation would strengthen the findings.

Small n. With $n = 8$ per cell, the interaction test for dose-response is underpowered ($p = 0.129$). The main effect ($p = 0.014$) is significant, but pairwise comparisons between adjacent cells are not.

No Partition B. The 50/50 partition was omitted; monotonicity of the dose-response gradient is assumed but not confirmed.

Treatment-C variance. The peer topology's best and worst individual-run scores both come from Treatment-C (composite 1.00 and 0.42). Peer coordination has a failure mode that hub-and-spoke does not.

Time confound. Treatment runs take 60–75% longer. We cannot fully disentangle quality improvement from additional exploration time, though the peer topology produces higher returns per additional minute than the control.

Simulated stakeholders. LLM-backed stakeholders may be more predictable than real humans, potentially advantaging the iterative follow-up process enabled by the treatment topology.

Transcript opacity. Agent Teams does not expose agent-level attribution for file operations, making indirect collaboration analysis unreliable. The true volume of inter-agent coordination is likely higher than our metrics

capture.

8 Conclusion

We conducted three experiments comparing symmetric peer agents against hub-and-spoke subagents on software engineering tasks. The first two experiments (bug-fixing and feature implementation) produced ceiling effects with zero inter-agent communication, establishing that code-level tasks with full information observability provide no structural advantage for peer coordination. The third experiment (architecture design with partitioned stakeholders) produced the first significant result: a large treatment effect ($d = +0.99$, $p = 0.014$) that scales with cross-partition conflict density (3.5× larger advantage at 75% cross-partition vs. 25%).

The most surprising finding is that agents never use direct peer messaging despite it being available. The treatment advantage arises from the topology's architectural properties: long-lived agents, shared file context, and coordinator-mediated iterative relay. This reframes the practical recommendation from "enable agent communication channels" to "design tasks with appropriate information distribution and use peer topologies for complex synthesis."

Future work should validate these findings across multiple scenarios and domains, increase sample sizes to achieve adequate power for interaction tests, introduce structured deliberation protocols to test whether explicit peer communication unlocks additional quality gains, and conduct human evaluation to rule out LLM-as-judge bias. Beyond the two-agent case studied here, a natural extension is to deeper agent hierarchies (sub-leads dispatching workers) and hybrid topologies (peers within clusters, hub-and-spoke between clusters), testing whether the information asymmetry mechanism scales or whether coordination overhead eventually dominates. Varying agent count (4, 8, 16) would clarify how the treatment effect interacts with the surface area of partition boundaries. The information asymmetry hypothesis—that it is the distribution of information, not the difficulty of the task, that determines whether multi-agent coordination improves quality—provides a testable framework for guiding these investigations.

References

[1] Nicholas Carlini. Building a C compiler with a team of parallel Claudes. `https://www.anthropic.com/engineering/building-c-compiler`, 2026.

[2] Anthropic. Claude code: Agent teams. `https://docs.anthropic.com/en/docs/claude-code`, 2026.

[3] Jacob Cohen. Statistical power analysis for the behavioral sciences. 1988.

[4] Jay R. Galbraith. Organization design: An information processing view. *Interfaces*, 4(3):28–36, 1974.

[5] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. *ICLR*, 2024.

[6] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating LLMs as agents. In *ICLR*, 2024.

[7] Henry B. Mann and Donald R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *Annals of Mathematical Statistics*, 18(1):50–60, 1947.

[8] João Moura. Crewai: Framework for orchestrating role-playing, autonomous AI agents. https://github.com/crewAIInc/crewAI, 2023.

[9] James D. Thompson. Organizations in action: Social science bases of administrative theory. 1967.

[10] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xu Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen LLM applications via multi-agent conversation. In *COLM*, 2024.

[11] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. *NeurIPS*, 2023.